Uruchamianie serwera Kafka4.x lokalnie:

… zostało opisane w pliku:

- https://github.com/vktech360/ApacheKafka
- https://www.youtube.com/watch?v=cxb9itmOPXI

Uruchamianie serwera Kafka lokalnie:

https://kafka.apache.org/40/documentation/compatibility.html?utm_source=chatgpt.com

JDK COMPATIBILITY ACROSS KAFKA VERSIONS

Module	Kafka Version	Java 11	Java 17	Java 23
Clients	4.0.0	✓	✓	✓
Streams	4.0.0	✓	✓	✓
Connect	4.0.0	✗	✓	✓
Server	4.0.0	✗	✓	✓

1. Apache Kafka 4.x wymaga : wersji java JDK 17

Sprawdzenie: → java -version

2. Pobieramy brokera Kafka ze strony: → https://kafka.apache.org/downloads

Zakładamy na dysku **c:** katalog: **md kafka4.1**

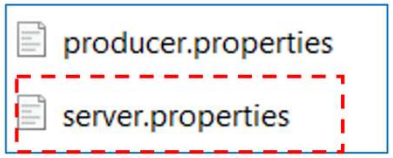
W katalogu tym należy rozpakować pobrane archiwum

3. Następnie dokonujemy jego konfiguracji:

- w katalogu: → **Kafka4.1** → należy założyć podkatalog: → **C:\kafka4.1\logs**

Dokonujemy edycji plików konfiguracyjnych: **server.properties** znajdujących się w : **C:\kafka4.1\config**

server.properties → **log.dirs=C:/kafka4.1/logs**



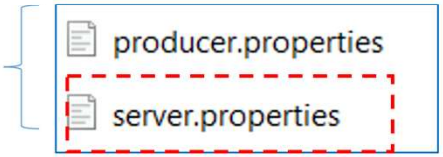

```
# A comma separated list of directories under which to store log files
# log.dirs=/tmp/kraft-combined-logs
log.dirs=c:/kafka4.1/logs
```

Uruchamianie serwera Kafka lokalnie:

5. W trybie **KRaft (Kafka Raft Metadata mode)** Kafka utrzymuje tzw. **quorum kontrolerów**

węzłów, które zarządzają metadanymi klastra (tematy, partycje, ACL, itp.), podobnie jak wcześniej robił to Zookeeper.

→ … następnie dodajemy w pliku konfiguracyjnym **server.properties**



… wpis:

controller.quorum.voters=1@localhost:9093

Składnia ogólna: controller.quorum.voters=<node.id1>@<host1>:<port1>,<node.id2>@<host2>:<port2>,...

Każdy element opisuje jeden kontroler (czyli broker, który może być kontrolerem) w formacie:

- node.id → unikalny identyfikator węzła w klastrze Kafka,
- host:port → adres, na którym kontroler komunikuje się z innymi węzłami (port kontrolny, nie ten sam co listeners brokera!).

```
# List of controller endpoints used connect to the controller cluster
controller.quorum.bootstrap.servers=localhost:9093

controller.quorum.voters=1@localhost:9093
```

Dla klastra z 3 kontrolerami byłoby np.: controller.quorum.voters=1@node1:9093, 2@node2:9093, 3@node3:9093

Uruchamianie serwera Kafka lokalnie:

5. Podsumowanie:

Element	Znaczenie
controller.quorum.voters	Lista członków klastra kontrolerów (KRaft)
1@localhost:9093	Identyfikator 1 i adres kontrolera (host + port)
Używany w trybie	**KRaft (bez Zookeepera)**
Typowa wartość w dev	1@localhost:9093
Typowa wartość w produkcji	1@host1:9093,2@host2:9093,3@host3:9093

Uruchamianie serwera Kafka lokalnie:

5. **Wygenerowania unikalnego identyfikatora (UUID)** klastra metadanych.

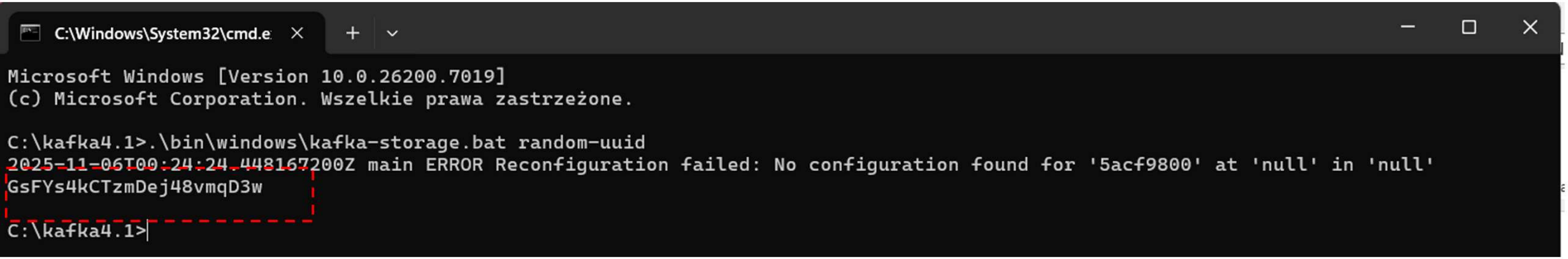
```
.\bin\windows\kafka-storage.bat random-uuid
```

Ten **UUID** to **cluster ID** → unikalny identyfikator klastra KRaft.

Kafka w trybie KRaft nie używa już Zookeepera, więc musi sama zarządzać metadanymi i rozpoznawać, które węzły należą do danego klastra.

Właśnie ten identyfikator pozwala brokerom rozpoznać:

- → że należą do tego samego klastra,
- → oraz że mają **ten sam dziennik metadanych (metadata log)**.



Występuje błąd → ale mamy klucz, który należy skopiować do notatnika

Uruchamianie serwera Kafka lokalnie:

5. **Wygenerowania unikalnego identyfikatora (UUID)** klastra metadanych.

.\bin\windows\kafka-storage.bat random-uuid

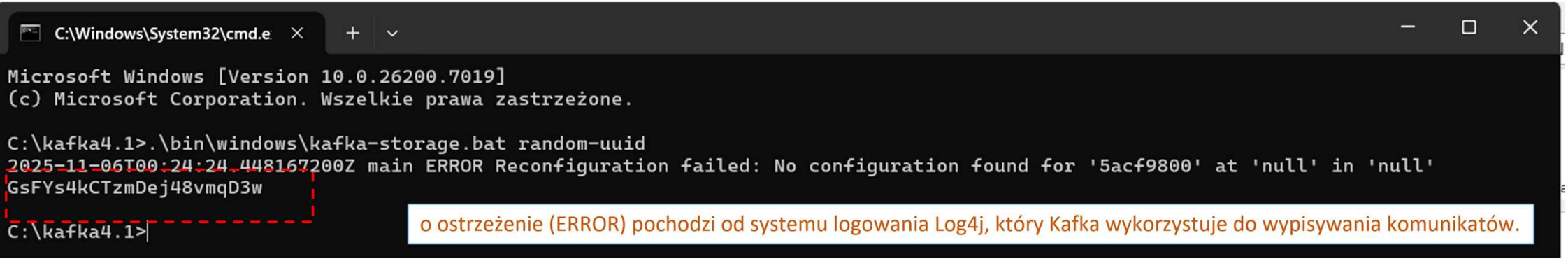
Ten **UUID** to **cluster ID** → unikalny identyfikator klastra KRaft.

Kafka w trybie KRaft nie używa już Zookeepera, więc musi sama zarządzać metadanymi i rozpoznawać, które węzły należą do danego klastra.

Właśnie ten identyfikator pozwala brokerom rozpoznać:

→ że należą do tego samego klastra,

→ oraz że mają **ten sam dziennik metadanych (metadata log)**.



o ostrzeżenie (ERROR) pochodzi od systemu logowania Log4j, który Kafka wykorzystuje do wypisywania komunikatów.

Występuje błąd z Log4j → ale mamy klucz, który należy skopiować do notatnika

Uruchamianie serwera Kafka lokalnie:

6. Następne polecenie sformatuje katalog danych i utworzy plik: **meta.properties** z Twoim identyfikatorem klastra.

```
.\bin\windows\kafka-storage.bat format -t <UUID> -c .\config\server.properties --ignore-formatted
```

Tu podmieniamy → **UUID** na nasz unikalny identyfikator klastra KRaft.

```
.\bin\windows\kafka-storage.bat format -t E7SRW44mSuqyglsQZV0AbQ -c .\config\server.properties --ignore-formatted
```

```
C:\kafka4.1>.\bin\windows\kafka-storage.bat format -t GsFYs4kCTzmDej48vmqD3w -c .\config\server.properties --ignore-formatted
2025-11-06T00:35:41.385554400Z main ERROR Reconfiguration failed: No configuration found for '5acf9800' at 'null' in 'null'
Formatting metadata directory c:/kafka4.1/logs with metadata.version 4.1-IV1.

C:\kafka4.1>
```

7. Następne uruchomić serwer Kafka w trybie Kraft:

Zmieniamy katalog: `cd C:\kafka4.1`

```
set KAFKA_HEAP_OPTS=-Xmx1G -Xms1G
```

Windows 11:

- set KAFKA_HEAP_OPTS ustawia rozmiar pamięci, który normalnie wmic miał obliczać.
- Dzięki temu, że zmienna już istnieje, skrypt pominie próby wywołania wmic.

```
.\bin\windows\kafka-server-start.bat .\config\server.properties
```

Wywołanie skryptu Apache Spark do nawiązania połączenia z brokerem Kafka:

https://www.youtube.com/watch?v=-fWznY5nBSg&ab_channel=TutorialsPedia

https://towardsdatascience.com/kafka-python-explained-in-10-lines-of-code-800e3e07dad1

(Jeżeli nie wystartowano serwera (Vide slajd poprzedni to go należy uruchomić

kafka-server-start.bat ..\..\config\server.properties

Zakładanie Topic:

kafka-topics.bat --create --topic **topicBD** --bootstrap-server localhost:9092

UWAGA!!

Producenta, który publikuje do szyny danych **topicBD** uruchomić w **cmd** z katalogu którym znajduje się plik **producer.py** → po zainstalowaniu zarówno w pythonie jak i pyCharmi-e biblioteki dla Kafki:

UWAGA!! Przy braku kompatybilności biblioteki:

pip install kafka-python

https://github.com/dpkp/kafka-python/issues/2412
1- Uninstall kafka-python using: **pip uninstall kafka-python**
2- Then run: **pip install --break-system-packages git+https://github.com/dpkp/kafka-python.git**

Starting a **PRODUCER** in Spark 4.x

D:\PycharmProjects\BigD → **python producer.py**

UWAGA!!

Pliki konfiguracyjne muszą być zapisane i zamknięte – inaczej usługi się nie uruchomią.

Usuwanie Topic:

1. Zatrzymać serwer Zookeper i Kafka oraz usunąć dane z:
   C:\kafka4.1\tmp\kafka-logs oraz **C:\kafka4.1\tmp\zookeeper**
2. Uruchomić serwer Zookeper i Kafka oraz ewentualnie usunąć Topic:
   kafka-topics.bat --delete --topic topicBD --bootstrap-server localhost:9092

Sprawdzenie działania Topic:

kafka-topics.bat --create --topic **test** --bootstrap-server localhost:9092

kafka-console-producer.bat --topic **test** --bootstrap-server localhost:9092

kafka-console-consumer.bat --topic **test** --bootstrap-server localhost:9092

UWAGA !!! → Skrypt uruchamiamy z pycharm-a → Komenda do znajduje się w postaci komentarza na górze pliku: *2_app.py*

Starting a **CONSUMER** in Spark 4.1

Spark ver 4.x: → *spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.2.0 2_app.py localhost:9092*